

Situação Finalizada**Iniciado** segunda-feira, 15 set. 2025, 16:01**Concluído** segunda-feira, 15 set. 2025, 16:33**Duração** 32 minutos 28 segundos**Nota** Ainda não avaliado**Questão 1**

Correto

Atingiu 1,00 de 1,00

Qual das alternativas abaixo melhor descreve a complexidade de espaço de um algoritmo?

- ☒ a. A quantidade de memória necessária para executar o algoritmo. ✓
- ☐ b. O tempo necessário para executar o algoritmo.
- ☐ c. A quantidade de instruções em linguagem de máquina.
- ☐ d. O número de operações realizadas pelo algoritmo.
- ☐ e. O tamanho do código-fonte do algoritmo.

Questão 2

Incorreto

Atingiu 0,00 de 2,00



Considere um algoritmo que utiliza um array de tamanho n e uma pilha auxiliar de tamanho $\log(n)$. Qual é a complexidade de espaço desse algoritmo?

- ☐ a. $O(n \log n)$
- ☒ b. $O(\log n)$ ✗
- ☐ c. $O(n)$
- ☐ d. $O(1)$

Questão 3

Correto

Atingiu 2,00 de 2,00

Dependendo da implementação do algoritmos de busca binária, marque as duas complexidades de espaço possíveis.

- ☐ a. $O(2!/2)$
- ☒ b. $O(1)$ ✓
- ☐ c. $O(n^2)$
- ☐ d. $O(n \log n)$
- ☐ e. $O(n)$
- ☒ f. $O(\log n)$ ✓
- ☐ g. $O(2^n)$
- ☐ h. $O(n^2/2)$

Questão 4

Correto

Atingiu 2,00 de 2,00

Qual das seguintes abordagens consome menos memória ao inverter uma lista ligada?

- ☐ a. Converter a lista em um array, inverter o array e recriar a lista.
- ☒ b. Alterar os ponteiros da lista original para inverter a ordem dos nós. ✓
- ☐ c. Utilizar uma pilha para armazenar os elementos e depois reconstruir a lista.
- ☐ d. Criar uma nova lista e copiar os elementos invertidos.
- ☐ e. Utilizar recursão para inverter os nós da lista.



Questão 5

Completo

Vale 3,00 ponto(s).

Apresente a análise da complexidade de espaço do algoritmo recursivo de fibonacci.

Faça uma análise descritiva apresentando a árvore de chamadas e explicando a sua conclusão.

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)
```

Para um $n = 10$, temos que:

```

      fib(3)
     /    \
  fib(2)  fib(1)
  /  \    /
fib(1) fib(0) 1
/    \
1      0    -

```

Fibonacci(3) = 2

Complexidade de espaço analisando a pilha de chamadas recursivas:

- empilha fib(3) tamanho da pilha = 1
- empilha fib(2) tamanho da pilha = 2
- empilha fib(1) tamanho da pilha = 3
- desempilha fib(1) tamanho da pilha = 2
- empilha fib(0) tamanho da pilha = 3
- desempilha fib(0) tamanho da pilha = 2
- desempilha fib(2) tamanho da pilha = 1
- empilha fib(1) tamanho da pilha = 2
- desempilha fib(1) tamanho da pilha = 1
- desempilha fib(3)

Como observado, a altura da pilha de chamadas nunca excedeu N . Portanto, mesmo que o número de entradas seja significativamente grande, a altura máxima da pilha de chamadas permanece em N . Isso implica que a complexidade do espaço é linear, denotada como $O(N)$.

